

vLLM greedy-generation benchmark

Date: 2026-07-12. This is a generation-only baseline for the V5 question-only teacher prompts. It deliberately does **not** request hidden states, run the teacher-forced hidden-state forward, write safetensors, or run the student premise forward.

Environment

The usable current environment is Fable's self-contained ``../venvs/vllm025``: vLLM 0.25.0+cu129, Torch 2.11.0+cu129, Python 3.12.10. It is compatible with the L40S driver because it links CUDA 12 libraries. The installation recipe and driver rationale are preserved in ``../howFableInstalledVLLMhere.md``.

The older comparison environment is ``../venvs/vllm126`` plus ``../2025/vllm``: vLLM 0.10.1rc2, Torch 2.8.0+cu128.

Protocol: 2,071 V5 window-RAG prompts; greedy decoding; the exact cache-builder per-record ceiling (``2 × estimated answer tokens + 96``); no truncation. The full workload permits 341,292 generated tokens, with a 932-token maximum ceiling. Peak memory is vLLM's physical-GPU reservation, sampled by ``nvidia-smi``.

L40S full-corpus results

Table batch: 64 prompts. All rows use one L40S and all 2,071 prompts. ``Load/setup`` is reported separately and includes weight load, KV-cache profiling and, in graph mode, compilation/capture. The ``peak VRAM`` measurement includes vLLM's KV reservation; it is not model-weight memory alone.

model	runtime / mode	load/setup	generation	generated tokens	tok/s	peak VRAM	hard cuts	next/prev LCS	cloze precision
--- --- ---: ---: ---: ---: ---: ---: ---: ---:									
Qwen3-0.6B	vLLM 0.10 eager	n/a	473.37 s	158,804	335.47	39.25 GiB	4.83%	51.41%	86.94%
Qwen3-0.6B	vLLM 0.25 eager	10.17 s	614.61 s	159,669	259.79	38.72 GiB	5.02%	52.33%	86.42%
Qwen3-0.6B	vLLM 0.25 graphs	123.22 s	**198.31 s**	158,918	**801.35**	39.20 GiB	4.68%	51.33%	86.75%
Qwen3-1.7B	vLLM 0.10 eager	n/a	567.40 s	169,180	298.17	38.77 GiB	n/a	59.28%	76.80%
Qwen3-1.7B	vLLM 0.25 graphs	123.88 s	477.60 s	168,511	352.83	38.93 GiB	12.17%	58.85%	77.22%
Qwen3-4B	vLLM 0.10 eager	n/a	980.37 s	165,115	168.42	38.71 GiB	n/a	72.96%	81.98%
Qwen3-4B	vLLM 0.25 graphs	123.95 s	899.26 s	164,550	182.98	39.19 GiB	15.79%	73.37%	82.04%
Qwen3-8B	vLLM 0.10 eager	n/a	1,472.14 s	130,878	88.90	39.45 GiB	n/a	76.44%	83.65%
Qwen3-8B	vLLM 0.25 graphs	124.38 s	1,420.77 s	131,607	92.63	39.53 GiB	6.04%	76.40%	83.88%

The small-model result is the clear win: after compilation, vLLM 0.25 graph mode is 4.04x faster than its own eager mode and 2.39x faster than vLLM 0.10 eager. At 1.7B--8B on this shared L40S host, graph mode only improves the old eager reference by 18%, 9%, and 5%, respectively. That scaling result is why the H100 measurements must be paired eager-versus-graphs tests, rather than a claim that the graph setting universally solves throughput.

This is **not** directly comparable to the approximately hour-scale old cache build: that build additionally performs the all-hidden-states teacher forward, CPU transfer and safetensors writes, and a student premise forward. The clean claim for 0.6B is that vLLM 0.25 graph mode produced all teacher answers in 3.3 minutes (5.4 minutes including a cold setup); it says nothing yet about the cache-build's non-generation phases.

Against the matching cached PyTorch greedy generations, vLLM 0.10 has 51.41% next/previous word recall versus 51.60%, 76.68 versus 76.41 mean answer tokens, and 72.38% exact decoded-answer agreement. It is therefore a valid inference substitute for this prompt/model combination.

Fable-build speed check: evenly spaced 256-prompt sample

Table batch: 64 prompts. All rows use Qwen3-0.6B and the same deterministic evenly spaced V5 sample, and report generation time only. Engine load/compilation is separate.

engine	mode	generation time	tokens	tok/s	peak VRAM	next/prev LCS	cloze precision	exact cache text	
---	---	---	---	---	---	---	---	---	
vLLM 0.10.1 + cu128	eager	68.47 s	19,202	280.46	39.25 GiB	54.44%	87.19%	69.14%	
vLLM 0.25 + cu129	eager	90.23 s	19,483	215.94	39.65 GiB	54.37%	88.28%	68.75%	
vLLM 0.25 + cu129	torch.compile + CUDA graphs	29.09 s	19,397	666.83	40.09 GiB	53.87%	91.32%	76.17%	

Interpretation: merely upgrading while forcing eager execution is 23% slower on this L40S sample. In its intended graph mode, the new build is 2.38× faster than the older eager engine. Its initial graph compilation takes about 41 s and must be amortized; the recorded `load\_seconds` includes setup while the table does not.

Eager batch-size control and PP check

The finished L40S 0.6B eager controls show why the production reference is batch 64: full-corpus B=16 took 925.23 s / 172.32 tok/s, versus B=64's 614.61 s / 259.79 tok/s. Both had approximately 10 s warm eager setup and 38.72 GiB

of vLLM reservation. The repeated GPU-2 256-prompt eager B=64 check was 93.16 s / 208.39 tok/s (10.20 s setup), consistent with the earlier 90.23 s result.

Qwen3-14B also completed a 256-prompt PP=2 graph-mode smoke test: 11,342 tokens in 318.90 s (35.57 tok/s), 174.02 s setup, and 38.31 GiB sampled on the first card. It establishes that pipeline parallelism runs, but it is not a single-card baseline nor a full-corpus measurement, and its first-card memory sample must not be reported as total PP memory.

Large-model scaling check

Qwen3-8B on the same 256-prompt batch-64 sample, using vLLM 0.25 graph-capable configuration but no forced graph result claim yet: 15,023 generated tokens in 194.06 s, or 77.41 tok/s, with 39.40 GiB peak VRAM, 6.25% hard cuts, 77.06% next/previous word recall, and 80.10% target-block lexical precision. The model's 15.27 GiB bf16 weights still fit on one L40S; usable KV capacity was 157,040 tokens.

H100 reproduction and next ladder

The H100 node is `agpuh01`, with DEV0/DEV1 reserved for this work; DEV2/DEV3 belong to another user and are never touched. Results go to `runs/vllm\_benchmark\_h100/` so the L40S evidence remains immutable.

Scoring correction. The historical `mean\_word\_acc` field (and every table column that called it “mean word recall”) is invalid: it computed `x.get("word\_acc", 0.0)`. Next and previous prompts carry a valid `word\_acc`; the 249 cloze prompts carry `containment` instead, so each was silently scored as zero. This is a cloze-only aggregation bug, not a next-versus-previous or pre-versus-post comparison. Historical aggregates are therefore removed below. For cloze, `containment` means the fraction of generated words that occur anywhere in the target block: target-block lexical precision. It is not cloze recall or blank-fill accuracy, because the deleted words were not stored. Every quality table reports it separately from next/previous reference-word recall; neither is an average of the other. The Qwen3-14B H100 graph audit's 85.32% task-mixed value remains a diagnostic only, not a recall claim or table entry.

The first H100 graph pair has completed with the exact L40S workload.

Table batch: 64 prompts. The H100 has 80 GiB HBM; at the same 0.85 vLLM reservation fraction it reserves roughly 70.6 GiB, so its reported peak is intentionally much larger than the L40S peak and does not mean the models need 70 GiB of weights.

	model	mode	load/setup	generation	generated tokens	tok/s	sampled VRAM	hard cuts	next/prev LCS	cloze precision	
	---	---	----	----	----	----	----	----	----	----	
	Qwen3-0.6B	vLLM 0.25 graphs	91.13 s	123.56 s	159,136	1,287.91	70.60 GiB	4.97%	52.06%	87.04%	
	Qwen3-0.6B	vLLM 0.25 eager	19.17 s	625.61 s	159,075	254.27	70.30 GiB	4.97%	52.05%	85.65%	
	Qwen3-1.7B	vLLM 0.25 graphs	89.54 s	194.37 s	169,142	870.19	70.59 GiB	12.41%	59.23%	76.90%	
	Qwen3-1.7B	vLLM 0.25 eager	19.39 s	709.25 s	169,081	238.39	70.30 GiB	12.12%	59.19%	77.26%	
	Qwen3-4B	vLLM 0.25 graphs	52.81 s	315.90 s	164,834	521.80	70.96 GiB	15.93%	73.41%	82.51%	
	Qwen3-4B	vLLM 0.25 eager	16.95 s	771.63 s	164,786	213.55	70.12 GiB	15.84%	73.45%	82.23%	
	Qwen3-8B	vLLM 0.25 graphs	60.66 s	421.97 s	130,514	309.30	70.65 GiB	6.23%	76.17%	83.63%	
	Qwen3-8B	vLLM 0.25 eager	19.79 s	705.38 s	130,501	185.01	70.06 GiB	6.04%	76.03%	84.06%	

This is the first important H100 finding: eager mode does not automatically use the H100's throughput. Relative to the same-mode L40S result, H100 eager is slightly slower at 0.6B (254 versus 260 tok/s) and still only 238 tok/s at 1.7B; CUDA graphs instead deliver 5.07x (0.6B), 3.65x (1.7B), 2.44x (4B), and 1.67x (8B) over H100 eager.

The completed 0.6B graph-mode full-corpus batch sweep demonstrates that the graph benefit is usable even at B=1, but throughput continues to scale through the H100-only B=128 point:

batch	generation time	generated tokens	tok/s	
----	----	----	----	
1	273.70 s	160,246	585.48	
2	255.96 s	159,267	622.22	
4	234.92 s	159,267	677.98	
8	204.97 s	159,267	777.01	
16	174.23 s	159,364	914.68	
32	146.78 s	159,882	1,089.24	
64	122.23 s	159,962	1,308.73	
128	107.85 s	160,172	**1,485.19**	

The paired 0.6B eager sweep on DEV1 completed. Its first rows show that eager mode scales only weakly with batch size on this workload:

batch	generation time	generated tokens	tok/s	
----	----	----	----	
1	1,444.93 s	158,917	109.98	
2	1,362.63 s	158,715	116.48	
4	1,216.63 s	158,778	130.51	
8	1,059.50 s	158,701	149.79	

The later fixed-32k capacity control provides the completed eager B=1--64 curve under an explicit long-context engine ceiling.

H100 larger-model and two-card throughput results

Table batch: 64 prompts (global, including PP2). These are the same 2,071-prompt, greedy, full-corpus protocol.

`sampled VRAM` is the highest reservation observed on either device, not the sum of both devices. The two GPT-OSS rows differ only in how the supplied passage is framed: the guided-memory wording asks the model to remember and recite the text, rather than presenting it as a formal retrieval task.

	model	placement / prompt framing	load/setup	generation	generated tokens	tok/s	sampled VRAM	hard cuts	next/prev LCS	cloze precision	
	---	---	---	---	---	---	---	---	---	---	
	Qwen3-14B	1 × H100, graphs	83.40 s	604.86 s	95,249	157.47	70.08 GiB	2.27%	84.18%	93.63%	
	Qwen3-32B	2 × H100 PP=2, graphs	105.20 s	1,568.65 s	135,463	86.36	73.16 GiB	5.21%	85.10%	74.90%	
	Qwen3.6-27B	2 × H100 PP=2, graphs	303.60 s	935.08 s	70,588	75.49	68.19 GiB	0.34%	97.93%	**98.10%**	
	Qwen3.6-35B-A3B	2 × H100 PP=2, graphs	258.07 s	284.83 s	74,496	261.55	65.60 GiB	0.19%	93.84%	97.13%	
	Gemma-4-31B-it	2 × H100 PP=2, graphs	219.09 s	1,158.31 s	78,296	67.59	71.11 GiB	0.24%	**98.91%**	75.04%	
	Gemma-4-26B-A4B-it	2 × H100 PP=2, graphs	182.28 s	298.71 s	85,038	**284.69**	65.50 GiB	1.45%	97.29%	81.37%	
	ALIA-40B-FC-2606	2 × H100 PP=2, graphs	109.14 s	1,501.85 s	94,482	62.91	68.63 GiB	3.04%	61.18%	90.15%	
	GPT-OSS-120B	2 × H100 PP=2, Harmony framing	116.23 s	505.45 s	180,091	372.13	69.74 GiB	21.78%	49.69%	76.42%	
	GPT-OSS-120B	2 × H100 PP=2, guided-memory framing	57.39 s	514.76 s	194,735	378.30	68.39 GiB	20.67%	51.52%	73.30%	

The table preserves the original purpose of this work: a speed baseline for our own cache builder. It separately records cold setup from answer decoding and deliberately excludes hidden-state extraction, GPU-to-CPU copies, safetensors writes, and the student premise forward. Those omitted phases are why it is a lower-bound baseline for V5 cache construction, not a claim that the full cache build should take the same time.

H100 additional one-card graph results

Table batch: 64 prompts. These are the same full-corpus protocol. They are preliminary graph-mode baselines; each receives a paired eager control later in the queue.

	model	load/setup	generation	generated tokens	tok/s	sampled VRAM	hard cuts	next/prev LCS	cloze precision	
	---	---	---	---	---	---	---	---	---	
	Phi-4	89.83 s	868.41 s	162,121	186.69	70.36 GiB	4.73%	93.18%	74.22%	

GPT-OSS-20B uses the Harmony memory framing. Its high decode speed does not make it a suitable recitation teacher in this condition: its hard-cut rate is 22.84% and its ordinary reference-word recall is 27.43%.

H100 completed one-card eager controls

Table batch: 64 prompts. The shared two-GPU queue completed these eager controls after the graph baselines. GPT-OSS-20B and Gemma-3-12B-it failed engine initialization once and were deliberately not retried. Nemotron's 99.71% hard-cut rate makes its speed result unsuitable as a normal recitation comparison.

model	load/setup	generation	generated tokens	tok/s	sampled VRAM	hard cuts	next/prev LCS	cloze precision
Qwen3-14B	40.09 s	652.33 s	94,832	145.37	70.10 GiB	2.12%	84.26%	94.27%
Phi-4	34.59 s	910.44 s	162,398	178.37	68.71 GiB	4.97%	93.11%	74.01%
NVIDIA Nemotron Nano 9B v2	78.09 s	2,088.06 s	328,697	157.42	70.05 GiB	99.71%	68.57%	22.13%
Llama 3.1 8B Instruct	23.35 s	612.81 s	137,739	224.77	69.72 GiB	5.41%	85.62%	65.32%
Mistral 7B Instruct v0.1	18.08 s	922.45 s	235,955	255.79	68.40 GiB	30.18%	57.82%	66.19%

Qwen3-14B is 1.08× faster with graphs (157.47 versus 145.37 tok/s), a much smaller gain than the 0.6B--8B Qwen3 graph controls. The completed controls therefore confirm that the graph benefit is model- and batch-shape-dependent, not simply a property of the accelerator.

H100 Qwen3.5 self-update ladder

Table batch: 64 prompts. All eight jobs completed in the shared queue. This is a family-relevant teacher ladder rather than a cross-family search: Qwen3.5 uses the Qwen continuation and its 250k vocabulary must be accounted for in any future frozen-vocabulary lens-loss experiment. Scores are the same separate next/previous LCS and cloze precision measures used throughout this document; neither column is a mixed recall average.

model	mode	load/setup	generation	generated tokens	tok/s	sampled VRAM	hard cuts	next/prev LCS	cloze precision
Qwen3.5-0.8B	graphs	115.93 s	177.79 s	243,586	**1,370.06**	68.01 GiB	57.36%	50.45%	65.44%
Qwen3.5-0.8B	eager	53.77 s	1,639.19 s	244,064	148.89	68.74 GiB	58.47%	50.53%	64.56%
Qwen3.5-2B	graphs	116.37 s	225.36 s	162,372	720.49	68.03 GiB	14.39%	62.02%	78.34%
Qwen3.5-2B	eager	48.85 s	1,545.96 s	162,983	105.43	69.09 GiB	14.87%	62.93%	78.26%

Qwen3.5-4B	graphs	111.21 s	254.17 s	86,670	341.00	68.24 GiB	2.22%	91.82%	92.56%	
Qwen3.5-4B	eager	52.67 s	1,240.15 s	86,683	69.90	69.31 GiB	2.03%	92.01%	92.77%	
Qwen3.5-9B	graphs	117.10 s	362.06 s	79,865	220.59	67.89 GiB	1.98%	93.05%	94.33%	
Qwen3.5-9B	eager	55.78 s	1,134.07 s	80,048	70.58	69.73 GiB	2.08%	93.24%	**94.39%**	

The graph/eager decode-speed ratios are 9.20×, 6.83×, 4.88×, and 3.13× from 0.8B through 9B. Quality is stable across execution mode at a given size; the salient quality transition is 2B to 4B, where hard cuts fall from about 14% to 2% and next/previous LCS rises from 62% to 92%.

H100 0.6B fixed-32k engine-capacity control

Context ceiling: 32,768 tokens; table batch is the `batch` column and is also vLLM `max\_num\_seqs`. This is a capacity-controlled repeat of the 0.6B full-corpus sweep, with a fresh engine at every point. All B=1--64 jobs fit; it measures performance beneath the fixed ceiling rather than finding an OOM boundary. The prompt workload itself remains the ordinary 2,071 V5 prompts, so the 32k ceiling changes engine capacity, not prompt length.

batch	graph tok/s	graph VRAM	eager tok/s	eager VRAM	
---: ---: ---: ---: ---:					
1	570.97	68.26 GiB	117.09	68.12 GiB	
2	605.08	68.26 GiB	121.28	68.12 GiB	
4	662.05	68.28 GiB	132.12	68.14 GiB	
8	757.01	68.28 GiB	152.86	68.14 GiB	
16	889.00	68.28 GiB	178.82	68.14 GiB	
32	1,056.35	68.28 GiB	215.03	68.14 GiB	
64	**1,284.24**	68.30 GiB	**259.09**	68.16 GiB	

Quality was stable over the sweep: graph next/previous LCS was 51.75--52.09% and cloze precision 85.77--87.00%; eager LCS was 51.72--51.97% and cloze precision 85.62--86.11%. At B=64 the 32k-capped graph result (1,284.24 tok/s) matches the original default-capacity graph result (1,287.91 tok/s), so the ordinary workload is not limited by this capacity cap.

Auxiliary: exporting a full hidden-state token to CPU (H100)

Teacher-cache egress is a different bottleneck from vLLM answer generation. For one answer-position token, a *full token* means its bf16 hidden vector at every transformer layer, i.e. `layers × hidden\_size × 2` bytes. The current cache writer (`TeacherCacheWriter.add`) transfers each layer independently via

``h.contiguous().cpu()`. The synthetic, weight-free H100 benchmark in `scripts/benchmark_hidden_transfer.py` measures that exact copy shape, without model forward, finite-value checks, safetensors serialization, or disk I/O.`

model	layers × hidden	full-token payload	bulk pinned copy, 1 token	current-like per-layer <code>`cpu()`</code> , 1 token	bulk pinned copy, 512 tokens	current-like per-layer <code>`cpu()`</code> , 512 tokens
--- ---: ---: ---: ---: ---: ---:						
Qwen3-0.6B	28 × 1,024	56 KiB	3.94 μs/token	359.75 μs/token	1.64 μs/token	23.78 μs/token
Qwen3-1.7B	28 × 2,048	112 KiB	4.64 μs/token	366.88 μs/token	2.09 μs/token	47.41 μs/token
Qwen3-4B	36 × 2,560	180 KiB	5.82 μs/token	470.35 μs/token	5.29 μs/token	71.96 μs/token
Qwen3-8B	36 × 4,096	288 KiB	8.00 μs/token	484.56 μs/token	8.51 μs/token	83.69 μs/token

The bulk pinned path reaches about 32--51 GiB/s at 512 tokens; the current-like path reaches only about 2.25--3.28 GiB/s because it allocates and synchronizes a pageable CPU transfer for every layer. Thus GPU→CPU bandwidth is not intrinsically the cache bottleneck when the full hidden-state tensor is batched; the present per-layer transfer pattern can become one. These figures are a transfer-only lower bound: the real cache writer also validates every CPU tensor and writes safetensors shards. Any writer change must preserve the per-layer on-disk schema and be measured end-to-end before adoption.

``nvidia-smi topo -m`` reports GPU0's CPU/NUMA affinity as cores 16--31 / NUMA node 1, while the benchmark process was unbound across all four NUMA nodes. The 32--51 GiB/s bulk range (34--55 GB/s) is therefore an end-to-end host-copy measurement, not a PCIe specification: its 51 GiB/s high point is consistent with a well-fed PCIe-5-class path, while the lower readings may include remote NUMA memory placement. A future exact link-limit measurement should bind both CPU execution and pinned-memory allocation to GPU0's NUMA node.

Only after those eight H100 reproductions finish, inspect cached larger model directories and benchmark models that fit on one or two H100s. For each, test pipeline parallelism first, then tensor parallelism only if time remains. These placement tests retain the same greedy answer-generation protocol and record per-device memory rather than claiming a single-card PP peak.

Repetition recipe

Use ``scripts/benchmark_vllm_generation.py``. A comparable H100 run must pin: model revision, V5 JSONL, tokenizer/chat template, generation allowance, batch size, ``max_model_len``, TP/PP layout, and whether CUDA graphs are enabled. Report load/compile time separately from generation time.

The next measurements are explicit 2-card and 4-card pipeline-parallel vLLM

runs. They should be treated as placement/throughput tests, not as a change to the teacher-generation target protocol.